

Real-world (rebuild a Project 1 feature, LCEL-style) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python, structured for reuse). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The mapping, and the two deliverables](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The mapping, and the two deliverables

Basic Version

You already built a structured-extraction feature in Project 1, using the raw OpenAI SDK. This exercise is not about changing that feature's job — it's about writing the *same* feature a second way, with LCEL, and keeping both.

Go find Project 1's structured-extraction code first. Read it again before writing anything new — you need to know exactly what input it takes and what output it returns, so your LCEL version matches.

Then think about how each raw-SDK step maps onto an LCEL piece: building messages → a prompt template; calling the API → `ChatOpenAI` ; reading `response_format` 's result → a structured output parser.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

This exercise has two deliverables: a new LCEL version of Project 1's structured-extraction feature (in a new file, not replacing the original), and a small script that runs both versions on the same 3 inputs and diffs the results.

The mapping from raw SDK to LCEL: your `messages` list becomes a `ChatPromptTemplate`; your `client.chat.completions.create(...)` call becomes a `ChatOpenAI` instance wired into a chain; your `response_format=YourPydanticModel` becomes `.with_structured_output(YourPydanticModel)`.

Reuse the same Pydantic model Project 1 already defines for its structured output — don't redefine it, import it; this is exactly what proves both versions produce the *same shape*, not just a similar-looking one. Keep `temperature=0` on both versions so any difference you see is from the method, not from randomness. Two Pydantic model instances with the same field values are equal with `==` by default — use that directly instead of writing manual field comparisons.

Once the basic loop prints a match per input, think about reuse: right now the comparison logic and the printing are tangled together in one script. Splitting them — a `run_comparison(test_inputs) -> list[ComparisonRow]` function that just computes, and a `main()` that just reports — means the exact same logic could later be called from a test, not just read by a human. This is the shape this document's Build Task needs.

Difference between Basic and Intermediate: Basic proves the two versions produce the same *answer* with printed output. Intermediate collects results into a typed structure and separates computing the comparison from reporting it, so the logic is reusable elsewhere.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
import Project 1's raw extraction function and its Pydantic model

build an LCEL chain using the same Pydantic model

make a list of 3 test inputs

for each input:
    run the raw version -> result A
    run the LCEL version -> result B
    print both
    print whether they match
```

Here is almost the whole thing — fill in your own project's real names:

```
# lcel_vs_raw_sdk_practice.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from project_1 import extract_raw, ExtractedData    # your real Project 1 names

prompt = ChatPromptTemplate.from_template("Extract structured data from: {text}")
lcel_chain = prompt | ChatOpenAI().with_structured_output(ExtractedData)
```

Expected output if you run just this: nothing yet — add the loop over your 3 test inputs, calling both `extract_raw` and `lcel_chain.invoke` .

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
def build_lcel_chain(model_name="gpt-4o-mini"):
    prompt = ChatPromptTemplate.from_template("...")
    model = ChatOpenAI(model=model_name, temperature=0)
    return prompt | model.with_structured_output(ExtractedData)

test_inputs = [input_1, input_2, input_3]

for text in test_inputs:
    raw_result = extract_raw(text)
    lcel_result = lcel_chain.invoke({"text": text})
    match = raw_result == lcel_result
    print(f"Input: {text}")
    print(f"  raw:  {raw_result}")
    print(f"  lcel: {lcel_result}")
    print(f"  match: {match}")
```

Write the full comparison script (the loop, printing both results and whether they match) yourself, then go one step further:

```
# lcel_vs_raw_sdk_practice.py
from dataclasses import dataclass

@dataclass
class ComparisonRow:
    input_text: str
    raw_result: "ExtractedData"
    lcel_result: "ExtractedData"
    matched: bool

def run_comparison(test_inputs: list[str]) -> list[ComparisonRow]:
    # your turn: build the lcel chain, loop over test_inputs, and
    # return a list of ComparisonRow instead of printing inline
    ...

def main() -> None:
    test_inputs = ["input one text", "input two text", "input three text"]
    rows = run_comparison(test_inputs)
    matched = sum(1 for row in rows if row.matched)
    print(f"{matched}/{len(rows)} matched")
```

```
for row in rows:
    if not row.matched:
        print(f"MISMATCH on: {row.input_text}")
```

Fill in `run_comparison()` yourself, then compare against the [Solution](#).

Difference between Basic and Intermediate: Basic prints a match line inline, in one script.

Intermediate collects results into a typed `ComparisonRow` list via `run_comparison()`, separate from `main()`'s reporting — the shape this document's Build Task needs.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)